The Journal of Open Source Software

torchTT: A PyTorch-based Tensor-Train Toolbox

Ion Gabriel Ion [1]

1 Terra Quantum AG

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Matt Graham

Reviewers:

- @dsuess
- @MarieRoald

Submitted: 16 January 2026
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

torchTT is a Python library enabling Tensor-Train (TT) decomposition (I. V. Oseledets, 2011) within the PyTorch framework (Paszke et al., 2019). It allows users to create and manipulate high-dimensional tensors in compressed TT format using PyTorch-like syntax and semantics. Operations such as basic linear algebra, matrix-vector products, and Kronecker products can be performed directly on the compressed representation without needing to form the full tensor. All computations leverage PyTorch's capabilities: TT instances live on the same device as their underlying data, facilitating seamless GPU acceleration, and automatic differentiation works through TT operations. Under the hood, torchTT exploits low-rank structure in high-dimensional arrays, dramatically reducing memory and computation compared to dense tensors. These features make torchTT especially suitable for problems with millions or billions of degrees of freedom where explicit storage is infeasible.

Statement of Need

High-dimensional tensors arise in many areas of science and engineering, such as the discretization of partial differential equations (PDEs), state spaces for systems of ordinary differential equations (ODEs), and many-body wavefunctions in quantum chemistry. Here, d denotes the number of tensor dimensions, also called modes (indices). In a tensor-product discretization, these modes correspond to the discretized variables, such as asset prices in multi-asset Black–Scholes PDEs or particle coordinates in many-body wavefunctions. Such problems can involve tens or hundreds of variables. A full tensor with n entries per mode contains n^d values: even $n = 100$ and $d = 10$ require 10^{20} values, or approximately 800 exabytes in double precision. This exponential growth is the curse of dimensionality.

TT typically approximates the full tensor using a sequence of smaller, three-dimensional cores, reducing storage to $\mathcal{O}(dnr^2)$ when the TT ranks are bounded by r. Accurate, compact approximations depend on suitable low-rank structure; arbitrary tensors need not admit small TT ranks. torchTT targets researchers and engineers in scientific computing, computational physics, and deep learning who need to work with such high-dimensional objects. It addresses the growing need for a single library that combines hardware-accelerated tensor network algebra with advanced numerical algorithms within the PyTorch ecosystem, enabling seamless integration into modern machine learning workflows.

State of the Field

Several open-source packages exist for tensor network computations and the TT format specifically. The ttpy 1.x series (I. Oseledets, 2021) provided AMEn (Alternating Minimal Energy) linear solvers and cross-approximation through CPU-based NumPy/Fortran routines, without native GPU acceleration. The subsequent ttpy 2 rewrite (I. Oseledets & ttpy contributors, 2026), introduced in 2026, adds NumPy and PyTorch backends with GPU support.

DRAFT

Libraries such as `tntorch` (Ballester-Ripoll, 2022), `t3f` (Novikov et al., 2020), and `TensorLy` (Kossaifi et al., 2019) have bridged the gap between tensor networks and machine learning frameworks by utilizing PyTorch and TensorFlow. These packages provide GPU acceleration and autograd capabilities, but offer different sets of numerical algorithms. For example, `tntorch` includes cross-approximation routines, while the documented APIs of these three packages do not include the AMEn solver for TT linear systems.

Additionally, in the domain of quantum physics, libraries such as `TeNPy` (Hauschild & Pollmann, 2018) provide highly optimized tools for simulating many-body systems using Matrix Product States (the physics equivalent of the TT format). However, these are deeply specialized for physics applications, such as the density matrix renormalization group (DMRG) algorithm, rather than general-purpose numerical mathematics or deep learning.

Development of `torchTT` began in 2021 to provide the combination of AMEn linear solves, cross-approximation, GPU execution, and PyTorch integration that was not available in the alternatives considered at that time. The package provides these numerical methods, including DMRG- and AMEn-based cross-approximation, alongside TT tensor and operator algebra and trainable TT layers through a common PyTorch-based representation and API.

Software Design

A central design trade-off in `torchTT` is balancing the expressiveness needed for advanced numerical algorithms with seamless integration into the PyTorch ecosystem. Two architectural choices support this integration.

Unified TT class and PyTorch-native design

The central component is the `torchtt.TT` class, which provides a single, unified representation for both TT-tensors and TT-matrices (tensor operators). This shared interface supports both scientific computing routines and models with TT-compressed parameters. Specialized algorithms such as AMEn and cross-approximation are implemented as functions in separate modules, separating algorithm-specific logic from the tensor representation. The class stores its TT cores as standard `torch.Tensor` objects, allowing PyTorch's autograd graph to track supported TT operations. This enables end-to-end gradient-based training of models that include TT-compressed layers or differentiable TT-based loss functions, without requiring custom backward passes.

- **Python-like API**: Operations such as addition (`+`), subtraction (`-`), elementwise multiplication (`*`), and matrix multiplication (`@`) are overloaded to work directly on `torchtt.TT` instances, keeping user code concise.
- **Device management**: Standard PyTorch methods like `.to(device)`, `.cuda()`, and `.cpu()` are supported, enabling TT objects and their computations to run on CPU or GPU without code changes.

Dual backend strategy

For performance-critical inner loops—particularly the AMEn solver—`torchTT` optionally delegates to a compiled C++ extension via PyTorch's custom operator mechanism. When the extension is unavailable, all algorithms fall back to a pure-PyTorch implementation with identical semantics. This dual-backend strategy ensures broad portability (any platform with PyTorch) while offering near-native performance when the C++ extension is compiled.

DRAFT

The Journal of Open Source Software

Functionality and Features

Beyond the core class, the `torchTT` library is organized into specialized modules targeting specific scientific and machine learning applications:

- **Linear System Solvers** (`torchtt.solvers`): Implements advanced solvers for linear systems $Ax = b$ directly in the TT format, including the AMEn method (Dolgov & Savostyanov, 2014). These solvers dynamically adapt the TT ranks during iteration to maintain accuracy. The module also includes elementwise division and inversion routines.
- **Cross Approximation** (`torchtt.interpolate`): Provides adaptive TT-cross interpolation methods (I. V. Oseledets & Tyrtyshnikov, 2010). This module constructs TT approximations of multivariate functions from black-box evaluations, essential for tasks where the tensor is too large to be formed explicitly but can be sampled.
- **Neural Network Layers** (`torchtt.nn`): Defines PyTorch-compatible neural network layers. `LinearLayerTT` parametrizes dense weight matrices as TT-matrices, enabling massive parameter reduction. `CompressedTTLayer` extends this concept by operating directly on TT-formatted inputs and applying nonlinear activations between TT cores during multiplication, targeting deep TT network architectures. All layers inherit from `torch.nn.Module`, making them drop-in replacements for standard layers in deep learning pipelines.
- **Manifold Optimization** (`torchtt.manifold`): Offers tools for Riemannian optimization on the manifold of tensors with fixed TT-rank. This includes projections onto the tangent space and Riemannian gradient calculation, facilitating advanced optimization tasks like tensor completion.

Example: Solving a Four-Dimensional PDE

The following example demonstrates how to solve a four-dimensional Poisson equation $\Delta u = f$ on $[0, 1]^4$ with zero boundary conditions using `torchTT`. The right-hand side f is constructed using cross-approximation, and the system is solved directly in the TT format using the AMEn solver. Additional runnable examples are provided in the repository's `examples/` folder.

```
import torchtt as tntt
import torch as tn

# Define the problem size
N, d = 64, 4

# Construct the one-dimensional Laplacian operator (finite difference)
L1d = (tn.diag(tn.ones(N-1),-1) + tn.diag(tn.ones(N-1),1) - \
      2*tn.eye(N)) / (1/(N-1))**2
L1d[0,1] = L1d[-1,-2] = 0 # Boundary conditions
L1d_tt = tntt.TT(L1d, shape=[(N,N)])

# Construct the d-dimensional Laplacian
L = L1d_tt ** tntt.eye([N]*3) + tntt.eye([N]) ** L1d_tt ** tntt.eye([N]*2) + \
    tntt.eye([N]*2) ** L1d_tt ** tntt.eye([N]) + tntt.eye([N]*3) ** L1d_tt

# Create the grid for cross approximation
x = tntt.meshgrid([tn.linspace(0,1,N)]*d)

# Approximate the RHS f(x) = x1(x1-1)*...*xd(xd-1)
f = tntt.interpolate.function_interpolate(lambda x: tn.prod(x * (x - 1), dim=1), x, eps=

# Solve the linear system Ax = f using AMEn on GPU
u = tntt.solvers.amen_solve(L.to('cuda'), f.to('cuda'), eps=1e-6)
```

DRAFT

Research Impact Statement

torchTT has demonstrated realized research impact across multiple domains, as evidenced by its use in peer-reviewed publications and ongoing collaborations:

- **High-Dimensional PDEs/ODEs**: torchTT can solve PDEs and dynamical systems in high dimensions by representing solution spaces and operators in TT form. Built-in solvers like AMEn make it practical to compute time steps of multi-dimensional PDEs, breaking the curse of dimensionality. Recent applications include isogeometric analysis for parameter-dependent geometries (Ion, Loukrezis, et al., 2022) and solving the chemical master equation for parameter inference (Ion, Wildner, et al., 2022).
- **Computational Physics and Chemistry**: The library enables efficient representation of high-dimensional objects like quantum many-body wavefunctions. Operators can be applied via TT-matrix-vector products, and observables computed via TT inner products, facilitating large-scale simulations. Furthermore, it supports analyzing entanglement scaling in Matrix Product State (MPS) representations and constructing shallow quantum circuits (Bohun et al., 2026).
- **Machine Learning with Compressed Models**: TT-layers enable significant compression of model parameters in deep learning. torchTT's compatibility with autograd allows for end-to-end training of compressed architectures and efficient on-device inference.
- **Data Science and Function Approximation**: The library is useful for surrogate modeling and uncertainty quantification, where high-dimensional response surfaces can be approximated adaptively using cross-approximation routines. Novel methods for local interpolation allow for constructing fine-scale TT representations from coarse-grid approximations with error guarantees (Guzman & Tiunov, 2026). This includes quantitative finance applications, where TT-cross approximation provides efficient option pricing surrogates for high-dimensional risk management (Gribben et al., 2026).

Acknowledgements

The authors thank the Terra Quantum AG team for their valuable discussions, feedback, and support.

AI Usage Disclosure

Artificial Intelligence tools were used to rephrase and refine the text of this paper. The core software implementation and the original draft of the content were produced by the authors.

References

Ballester-Ripoll, R. (2022). Tntorch: Tensor network learning with PyTorch. In *GitHub repository*. GitHub. https://github.com/rballester/tntorch

Bohun, V., Lukin, I., Luhanko, M., Korpas, G., De Brouwer, P. J. S., Maksymenko, M., & Koch-Janusz, M. (2026). Entanglement scaling in matrix product state representation of smooth functions and their shallow quantum circuit approximations. *Physical Review Research*, *8*(2), 023062. https://doi.org/10.1103/yyvr-dtsb

Dolgov, S. V., & Savostyanov, D. V. (2014). Alternating minimal energy methods for linear systems in higher dimensions. *SIAM Journal on Scientific Computing*, *36*(5), A2248–A2271. https://doi.org/10.1137/140953289

Gribben, D., Allende, C., Villarino, A., Cortines, A., Ali, M., Orús, R., Oswald, P., & Lehdili, N. (2026). STN-GPR: A singularity tensor network framework for efficient option pricing. *arXiv Preprint arXiv:2603.26318*. https://arxiv.org/abs/2603.26318

DRAFT

Guzman, J., & Tiunov, I. (2026). Local interpolation via low-rank tensor trains. *arXiv Preprint arXiv:2601.03885*. https://arxiv.org/abs/2601.03885

Hauschild, J., & Pollmann, F. (2018). Efficient numerical simulations with tensor networks: Tensor network python (TeNPy). *SciPost Physics Lecture Notes*, 005. https://doi.org/10.21468/SciPostPhysLectNotes.5

Ion, I. G., Loukrezis, D., & De Gersem, H. (2022). Tensor train based isogeometric analysis for PDE approximation on parameter dependent geometries. *Computer Methods in Applied Mechanics and Engineering*, *401*, 115593. https://doi.org/10.1016/j.cma.2022.115593

Ion, I. G., Wildner, C., Loukrezis, D., Koeppl, H., & De Gersem, H. (2022). Tensor-train approximation of the chemical master equation and its application for parameter inference. *MATHMOD 2022 Discussion Contribution Volume*, 39–40. https://doi.org/10.11128/arep.17.a17094

Kossaifi, J., Panagakis, Y., Anandkumar, A., & Pantic, M. (2019). TensorLy: Tensor learning in Python. *Journal of Machine Learning Research*, *20*(26), 1–6. http://jmlr.org/papers/v20/18-277.html

Novikov, A., Izmailov, P., Khrulkov, V., Figurnov, M., & Oseledets, I. (2020). Tensor train decomposition on TensorFlow (T3F). *Journal of Machine Learning Research*, *21*(30), 1–7.

Oseledets, I. (2021). Ttpy: Python implementation of the tensor train (TT)-toolbox. In *GitHub repository*. GitHub. https://github.com/oseledets/ttpy/tree/ttpy1

Oseledets, I. V. (2011). Tensor-train decomposition. *SIAM Journal on Scientific Computing*, *33*(5), 2295–2317. https://doi.org/10.1137/090752286

Oseledets, I. V., & Tyrtyshnikov, E. (2010). TT-cross approximation for multidimensional arrays. *Linear Algebra and Its Applications*, *432*(1), 70–88. https://doi.org/10.1016/j.laa.2009.07.024

Oseledets, I., & ttpy contributors. (2026). ttpy 2: Tensor Train toolbox in Python. In *GitHub repository*. GitHub. https://github.com/oseledets/ttpy/tree/0bd6622d9c425e6e98a2cd720c493e763225a4d6

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., … Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems 32* (pp. 8024–8035). Curran Associates, Inc. http://papers.nips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library.pdf

DRAFT